TanStack Query (React Query) Full Guide

TanStack Query (earlier React Query) is a powerful data-fetching and state management library for React. It helps you fetch, cache, synchronize, and update server state in your React applications efficiently.

■ Why TanStack Query?

- Managing API data with useEffect + useState is messy.
- Manual handling is required for loading, error, caching, and refetching.
- Server state changes outside your app, so local state is not enough.
- TanStack Query solves this by providing caching, deduplication, background refetching, pagination, mutation handling, and Devtools.

■ Installation

```
npm install @tanstack/react-query
npm install @tanstack/react-query-devtools
```

■■ Setup

```
import React from "react";
import ReactDOM from "react-dom/client";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import App from "./App";

const queryClient = new QueryClient();

ReactDOM.createRoot(document.getElementById("root")).render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>
);
```

■ Fetching Data with useQuery

```
import { useQuery } from "@tanstack/react-query";
import axios from "axios";

function Users() {
  const { data, error, isLoading, isError } = useQuery({
    queryKey: ["users"],
    queryFn: async () => {
      const res = await axios.get("https://jsonplaceholder.typicode.com/users");
      return res.data;
    },
  });

  if (isLoading) return <p>Loading...</p>;
  if (isError) return <p>Error: {error.message}</p>;
  return (
```

```
    <ul>
      {data.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

✍■ Mutations (useMutation)

```
import { useMutation, useQueryClient } from "@tanstack/react-query";
import axios from "axios";

function AddUser() {
  const queryClient = useQueryClient();

  const mutation = useMutation({
    mutationFn: (newUser) =>
      axios.post("https://jsonplaceholder.typicode.com/users", newUser),
    onSuccess: () => {
      queryClient.invalidateQueries(["users"]);
    },
  });

  return (
    <button
      onClick={() =>
        mutation.mutate({ name: "New User", email: "new@user.com" })
      }
    >
      Add User
    </button>
  );
}
```

■ Common Interview Questions and Answers

Q: What is TanStack Query, and how is it different from Redux?

A: TanStack Query is focused on server state management (fetching, caching, updating API data), while Redux is a global state container for both local and server state. TanStack Query automatically handles caching, retries, and refetching, which Redux does not.

Q: Explain the difference between useQuery and useMutation.

A: useQuery is for fetching and caching GET requests, while useMutation is for POST, PUT, DELETE (write) operations. Mutations can invalidate queries to trigger refetching.

Q: How does caching work in TanStack Query?

A: Data fetched via useQuery is stored in an in-memory cache. When the same queryKey is used again, cached data is served instantly and updated in the background.

Q: What is the purpose of queryKey in useQuery?

A: queryKey uniquely identifies queries. It is used for caching, invalidation, and refetching. Example: ['users'] or ['user', userId].

Q: How can you handle pagination or infinite scrolling with TanStack Query?

A: TanStack Query provides useInfiniteQuery hook for infinite scroll and cursor-based pagination. useQuery can also fetch paginated data using page parameters in the queryKey.

Q: What are stale queries and how does TanStack Query handle them?

A: Queries become 'stale' after a set staleTime. Stale queries are refetched in the background when components mount or when window refocuses.

Q: How do you invalidate queries after a mutation?

A: By calling queryClient.invalidateQueries(['queryKey']), which refetches the affected data.

Q: What advantages do TanStack Query Devtools provide?

A: They show query cache state, loading, error, retries, background refetches, and help debug query lifecycle easily.

Q: How does React Query handle retries and error handling?

A: By default, it retries failed requests 3 times with exponential backoff. You can customize retry count, delay, and error handling with options.

Q: When would you choose TanStack Query over traditional state management libraries like Redux or Context API?

A: When your app heavily depends on remote server data (APIs), TanStack Query is preferred as it simplifies caching, synchronization, and background updates. For pure client-side state, Redux/Context might still be useful.